

Running Custom Playwright Code

Use run-code to execute arbitrary Playwright code for advanced scenarios not covered by CLI commands.

Syntax

```
playwright-cli run-code "async page => {  
  // Your Playwright code here  
  // Access page.context() for browser context operations  
}"
```

You can also load the function from a file:

```
playwright-cli run-code --filename=./my-script.js
```

The code must be a single function expression, it is wrapped in (...) and evaluated. import/export/require syntax is not supported.

Geolocation

Grant geolocation permission and set location

```
playwright-cli run-code "async page => {  
  await page.context().grantPermissions(['geolocation']);  
  await page.context().setGeolocation({ latitude: 37.7749, longitude: -122.4194  
}"
```

Set location to London

```
playwright-cli run-code "async page => {  
  await page.context().grantPermissions(['geolocation']);  
  await page.context().setGeolocation({ latitude: 51.5074, longitude: -0.1278 })  
}"
```

Clear geolocation override

```
playwright-cli run-code "async page => {  
  await page.context().clearPermissions();  
}"
```

Permissions

Grant multiple permissions

```
playwright-cli run-code "async page => {  
  await page.context().grantPermissions([  
    'geolocation',  
    'notifications',  
    'camera',
```

```

        'microphone'
    });
}"

# Grant permissions for specific origin
playwright-cli run-code "async page => {
  await page.context().grantPermissions(['clipboard-read'], {
    origin: 'https://example.com'
  });
}"

```

Media Emulation

```

# Emulate dark color scheme
playwright-cli run-code "async page => {
  await page.emulateMedia({ colorScheme: 'dark' });
}"

# Emulate light color scheme
playwright-cli run-code "async page => {
  await page.emulateMedia({ colorScheme: 'light' });
}"

# Emulate reduced motion
playwright-cli run-code "async page => {
  await page.emulateMedia({ reducedMotion: 'reduce' });
}"

# Emulate print media
playwright-cli run-code "async page => {
  await page.emulateMedia({ media: 'print' });
}"

```

Wait Strategies

```

# Wait for network idle
playwright-cli run-code "async page => {
  await page.waitForLoadState('networkidle');
}"

# Wait for specific element
playwright-cli run-code "async page => {
  await page.locator('.loading').waitFor({ state: 'hidden' });
}"

```

```

# Wait for function to return true
playwright-cli run-code "async page => {
  await page.waitForFunction(() => window.appReady === true);
}"

# Wait with timeout
playwright-cli run-code "async page => {
  await page.locator('.result').waitFor({ timeout: 10000 });
}"

```

Frames and Iframes

```

# Work with iframe
playwright-cli run-code "async page => {
  const frame = page.locator('iframe#my-iframe').contentFrame();
  await frame.locator('button').click();
}"

# Get all frames
playwright-cli run-code "async page => {
  const frames = page.frames();
  return frames.map(f => f.url());
}"

```

File Downloads

```

# Handle file download
playwright-cli run-code "async page => {
  const downloadPromise = page.waitForEvent('download');
  await page.getByRole('link', { name: 'Download' }).click();
  const download = await downloadPromise;
  await download.saveAs('./downloaded-file.pdf');
  return download.suggestedFilename();
}"

```

Clipboard

```

# Read clipboard (requires permission)
playwright-cli run-code "async page => {
  await page.context().grantPermissions(['clipboard-read']);
  return await page.evaluate(() => navigator.clipboard.readText());
}"

# Write to clipboard
playwright-cli run-code "async page => {

```

```
    await page.evaluate(text => navigator.clipboard.writeText(text), 'Hello clipboard!');
  }"
```

Page Information

```
# Get page title
playwright-cli run-code "async page => {
  return await page.title();
}"
```

```
# Get current URL
playwright-cli run-code "async page => {
  return page.url();
}"
```

```
# Get page content
playwright-cli run-code "async page => {
  return await page.content();
}"
```

```
# Get viewport size
playwright-cli run-code "async page => {
  return page.viewportSize();
}"
```

JavaScript Execution

```
# Execute JavaScript and return result
playwright-cli run-code "async page => {
  return await page.evaluate(() => {
    return {
      userAgent: navigator.userAgent,
      language: navigator.language,
      cookiesEnabled: navigator.cookieEnabled
    };
  });
}"
```

```
# Pass arguments to evaluate
playwright-cli run-code "async page => {
  const multiplier = 5;
  return await page.evaluate(m => document.querySelectorAll('li').length * m, multiplier);
}"
```

Error Handling

```
# Try-catch in run-code
playwright-cli run-code "async page => {
  try {
    await page.getByRole('button', { name: 'Submit' }).click({ timeout: 1000 });
    return 'clicked';
  } catch (e) {
    return 'element not found';
  }
}"
```

Complex Workflows

```
# Login and save state
playwright-cli run-code "async page => {
  await page.goto('https://example.com/login');
  await page.getByRole('textbox', { name: 'Email' }).fill('user@example.com');
  await page.getByRole('textbox', { name: 'Password' }).fill('secret');
  await page.getByRole('button', { name: 'Sign in' }).click();
  await page.waitForURL('**/dashboard');
  await page.context().storageState({ path: 'auth.json' });
  return 'Login successful';
}"
```

```
# Scrape data from multiple pages
playwright-cli run-code "async page => {
  const results = [];
  for (let i = 1; i <= 3; i++) {
    await page.goto(`https://example.com/page/${i}`);
    const items = await page.locator('.item').allTextContents();
    results.push(...items);
  }
  return results;
}"
```